

****Schema (MySQL v8)****

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    manager_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    amount DECIMAL(10,2),
```

```
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

```
INSERT INTO departments VALUES
```

```
(1,'Engineering'),
```

```
(2,'Data'),
```

```
(3,'Finance'),
```

```
(4,'HR'),
```

```
(5,'Sales');
```

```
INSERT INTO employees VALUES
```

```
(101,'John',1,120000,'2018-01-15',NULL),
```

```
(102,'Mike',1,95000,'2019-03-10',101),
```

```
(103,'David',1,90000,'2021-06-20',101),
```

```
(104,'Sarah',2,140000,'2017-08-25',NULL),
```

```
(105,'Priya',2,110000,'2020-02-14',104),
```

```
(106,'Ravi',2,105000,'2022-01-10',104),
```

```
(107,'Anil',2,85000,'2023-05-12',105),
```

```
(108,'Emma',3,125000,'2016-11-18',NULL),
```

```
(109,'Robert',3,85000,'2020-04-21',108),
```

```
(110,'Sophia',4,95000,'2021-09-11',NULL),
```

```
(111,'Chris',4,65000,'2024-01-15',110),
```

```
(112,'James',5,130000,'2018-12-05',NULL),
```

```
(113,'Linda',5,100000,'2019-10-10',112),
```

```
(114,'Kevin',5,95000,'2022-03-15',112),  
(115,'Nancy',5,75000,'2024-06-01',113);
```

```
INSERT INTO customers VALUES
```

```
(1,'Amazon','Hyderabad'),  
(2,'Google','Bangalore'),  
(3,'Microsoft','Hyderabad'),  
(4,'Netflix','Mumbai'),  
(5,'Meta','Delhi'),  
(6,'Adobe','Pune'),  
(7,'Amazon','Hyderabad');
```

```
INSERT INTO orders VALUES
```

```
(1001,1,'2024-01-05',25000),  
(1002,2,'2024-01-10',45000),  
(1003,1,'2024-02-02',30000),  
(1004,3,'2024-02-15',55000),  
(1005,2,'2024-03-01',70000),  
(1006,4,'2024-03-18',15000),  
(1007,1,'2024-04-10',45000),  
(1008,5,'2024-04-25',85000),  
(1009,3,'2024-05-05',35000),  
(1010,6,'2024-05-20',40000),  
(1011,2,'2024-06-10',95000),  
(1012,5,'2024-06-15',25000),  
(1013,1,'2024-07-01',60000),  
(1014,4,'2024-07-08',20000),
```

```
(1015,2,'2024-08-10',55000),  
(1016,3,'2024-08-15',65000),  
(1017,5,'2024-09-01',45000),  
(1018,1,'2024-09-20',50000),  
(1019,6,'2024-10-05',30000),  
(1020,2,'2024-10-22',80000);
```

****Query #1****

-- Query 1: Display employee name, department name, and salary.

```
SELECT e.emp_name, d.dept_name, e.salary  
FROM employees e  
JOIN departments d ON e.dept_id = d.dept_id;
```

```
| emp_name | dept_name | salary |  
| ----- | ----- | ----- |  
| John    | Engineering | 120000.0 |  
| Mike    | Engineering | 95000.0 |  
| David   | Engineering | 90000.0 |  
| Sarah   | Data       | 140000.0 |  
| Priya   | Data       | 110000.0 |  
| Ravi    | Data       | 105000.0 |  
| Anil    | Data       | 85000.0 |  
| Emma    | Finance    | 125000.0 |  
| Robert  | Finance    | 85000.0 |  
| Sophia  | HR         | 95000.0 |
```

Chris	HR	65000.0	
James	Sales	130000.0	
Linda	Sales	100000.0	
Kevin	Sales	95000.0	
Nancy	Sales	75000.0	

****Query #2****

-- Query 2: Show all departments and the number of employees in each department.

```
SELECT dept_name,COUNT(e.emp_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.dept_id=e.dept_id
GROUP BY d.dept_name;
```

dept_name	emp_count	
-----	-----	
Engineering	3	
Data	4	
Finance	2	
HR	2	
Sales	4	

****Query #3****

-- Query 3: Find employees along with their manager names.

```
SELECT e.emp_name AS employee_name,m.emp_name AS manager_name
```

```
FROM employees e
LEFT JOIN employees m ON e.manager_id=m.emp_id;
```

```
| employee_name | manager_name |
| ----- | ----- |
| John      |      |
| Mike      | John  |
| David      | John  |
| Sarah      |      |
| Priya      | Sarah |
| Ravi       | Sarah |
| Anil       | Priya |
| Emma       |      |
| Robert     | Emma  |
| Sophia     |      |
| Chris      | Sophia|
| James      |      |
| Linda      | James |
| Kevin      | James |
| Nancy      | Linda |
```

****Query #4****

-- Query 4: Find the total salary expenditure for each department.

```
SELECT d.dept_id,d.dept_name,SUM(e.salary) AS sum_salary
FROM employees e
JOIN departments d ON e.dept_id=d.dept_id
```

```
GROUP BY d.dept_id,d.dept_name;
```

```
| dept_id | dept_name | sum_salary |
| ----- | -
```

dept_id	dept_name	sum_salary
1	Engineering	305000.0
2	Data	440000.0
3	Finance	210000.0
4	HR	160000.0
5	Sales	400000.0

****Query #5****

-- Query 5: Find the average salary for each department.

```
SELECT d.dept_id,d.dept_name,AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id=d.dept_id
GROUP BY d.dept_id,d.dept_name
ORDER BY avg_salary DESC;
```

```
| dept_id | dept_name | avg_salary |
| ----- | -
```

dept_id	dept_name	avg_salary
2	Data	110000.0
3	Finance	105000.0
1	Engineering	101666.666667
5	Sales	100000.0
4	HR	80000.0

****Query #6****

-- Query 6: Find the department with the highest average salary.

```
SELECT d.dept_id,d.dept_name,AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id=d.dept_id
GROUP BY d.dept_id,d.dept_name
ORDER BY avg_salary DESC
LIMIT 1;
```

```
| dept_id | dept_name | avg_salary |
| ----- | - | - |
| 2      | Data      | 110000.0   |
```

****Query #7****

-- Query 7: Find employees earning more than the company average salary.

```
SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
);
```

```
| emp_name | salary |
| - | - |
```


John	120000.0	
Sarah	140000.0	
Priya	110000.0	
Ravi	105000.0	
Emma	125000.0	
James	130000.0	

****Query #8****

-- Query 8: Find employees earning more than their department average salary.

```

SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e.dept_id
);

```

emp_name	salary	
-----	-----	
John	120000.0	
Sarah	140000.0	
Emma	125000.0	
Sophia	95000.0	
James	130000.0	

****Query #9****

-- Query 9: Find the second highest salary in the company.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

```
| second_highest_salary |
| ----- |
| 130000.0 |
```

****Query #10****

-- Query 10: Find employees who earn the maximum salary in their department.

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e.dept_id
);
```

```
| emp_name | dept_id | salary |
| ----- | ----- | ----- |
```

John	1	120000.0
Sarah	2	140000.0
Emma	3	125000.0
Sophia	4	95000.0
James	5	130000.0

****Query #11****

-- Query 11: Rank employees based on salary within each department.

```
SELECT e.emp_id, e.emp_name, e.dept_id, e.salary,
       RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS salary_rank
FROM employees e;
```

emp_id	emp_name	dept_id	salary	salary_rank	
-----	-----	-----	-----	-----	
101	John	1	120000.0	1	
102	Mike	1	95000.0	2	
103	David	1	90000.0	3	
104	Sarah	2	140000.0	1	
105	Priya	2	110000.0	2	
106	Ravi	2	105000.0	3	
107	Anil	2	85000.0	4	
108	Emma	3	125000.0	1	
109	Robert	3	85000.0	2	
110	Sophia	4	95000.0	1	
111	Chris	4	65000.0	2	
112	James	5	130000.0	1	

113	Linda	5	100000.0	2	
114	Kevin	5	95000.0	3	
115	Nancy	5	75000.0	4	

****Query #12****

-- Query 12: Find the top 3 highest-paid employees in each department.

```

SELECT * FROM (
    SELECT e.emp_id, e.emp_name, e.dept_id, e.salary,
           DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS
salary_rank
    FROM employees e
) AS t
WHERE salary_rank <= 3;

```

emp_id	emp_name	dept_id	salary	salary_rank	
-----	-----	-----	-----	-----	
101	John	1	120000.0	1	
102	Mike	1	95000.0	2	
103	David	1	90000.0	3	
104	Sarah	2	140000.0	1	
105	Priya	2	110000.0	2	
106	Ravi	2	105000.0	3	
108	Emma	3	125000.0	1	
109	Robert	3	85000.0	2	
110	Sophia	4	95000.0	1	
111	Chris	4	65000.0	2	

112	James	5	130000.0	1	
113	Linda	5	100000.0	2	
114	Kevin	5	95000.0	3	

****Query #13****

-- Query 13: Calculate running total salary ordered by hire date.

```
SELECT e.emp_id, e.emp_name, e.hire_date, e.salary,
       SUM(e.salary) OVER (ORDER BY e.hire_date) AS running_total_salary
FROM employees e
ORDER BY e.hire_date;
```

emp_id	emp_name	hire_date	salary	running_total_salary	
-----	-----	-----	-----	-----	
108	Emma	2016-11-18	125000.0	125000.0	
104	Sarah	2017-08-25	140000.0	265000.0	
101	John	2018-01-15	120000.0	385000.0	
112	James	2018-12-05	130000.0	515000.0	
102	Mike	2019-03-10	95000.0	610000.0	
113	Linda	2019-10-10	100000.0	710000.0	
105	Priya	2020-02-14	110000.0	820000.0	
109	Robert	2020-04-21	85000.0	905000.0	
103	David	2021-06-20	90000.0	995000.0	
110	Sophia	2021-09-11	95000.0	1090000.0	
106	Ravi	2022-01-10	105000.0	1195000.0	
114	Kevin	2022-03-15	95000.0	1290000.0	
107	Anil	2023-05-12	85000.0	1375000.0	

111	Chris	2024-01-15	65000.0	1440000.0	
115	Nancy	2024-06-01	75000.0	1515000.0	

****Query #14****

-- Query 14: Show previous employee salary using LAG().

```
SELECT e.emp_id, e.emp_name, e.salary,
       LAG(e.salary) OVER (ORDER BY e.hire_date) AS previous_salary
FROM employees e
ORDER BY e.hire_date;
```

emp_id	emp_name	salary	previous_salary	
-----	-----	-----	-----	
108	Emma	125000.0		
104	Sarah	140000.0	125000.0	
101	John	120000.0	140000.0	
112	James	130000.0	120000.0	
102	Mike	95000.0	130000.0	
113	Linda	100000.0	95000.0	
105	Priya	110000.0	100000.0	
109	Robert	85000.0	110000.0	
103	David	90000.0	85000.0	
110	Sophia	95000.0	90000.0	
106	Ravi	105000.0	95000.0	
114	Kevin	95000.0	105000.0	
107	Anil	85000.0	95000.0	
111	Chris	65000.0	85000.0	

115	Nancy	75000.0	65000.0	
-----	-------	---------	---------	--

****Query #15****

-- Query 15: Find customers who never placed an order.

```
SELECT c.customer_id, c.customer_name, c.city
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

customer_id	customer_name	city	
-----	-----	-----	
7	Amazon	Hyderabad	

****Query #16****

-- Query 16: Find the latest order for each customer.

```
SELECT customer_id, order_id, order_date, amount
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY o.order_date DESC)
    AS rn
    FROM orders o
) AS t
WHERE rn = 1;
```

customer_id	order_id	order_date	amount
1	1018	2024-09-20	50000.0
2	1020	2024-10-22	80000.0
3	1016	2024-08-15	65000.0
4	1014	2024-07-08	20000.0
5	1017	2024-09-01	45000.0
6	1019	2024-10-05	30000.0

****Query #17****

-- Query 17: Find total sales generated by each customer.

```

SELECT c.customer_name,SUM(o.amount) AS total_sales
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
GROUP BY c.customer_name;

```

customer_name	total_sales
Amazon	210000.0
Google	345000.0
Microsoft	155000.0
Netflix	35000.0
Meta	155000.0
Adobe	70000.0

****Query #18****

-- Query 18: Calculate customer contribution percentage to overall sales.

```
SELECT c.customer_name,  
       SUM(o.amount) AS customer_sales,  
       ROUND(SUM(o.amount) * 100.0 / (SELECT SUM(amount) FROM orders), 2) AS  
contribution_percentage  
  
FROM customers c  
  
JOIN orders o ON c.customer_id = o.customer_id  
  
GROUP BY c.customer_name;
```

customer_name	customer_sales	contribution_percentage
-----	-----	-----
Amazon	210000.0	21.65
Google	345000.0	35.57
Microsoft	155000.0	15.98
Netflix	35000.0	3.61
Meta	155000.0	15.98
Adobe	70000.0	7.22

****Query #19****

-- Query 19: Identify duplicate customer names.

```
SELECT c.customer_name, COUNT(*) AS duplicate_count  
  
FROM customers c  
  
GROUP BY c.customer_name  
  
HAVING COUNT(*) > 1;
```

customer_name	duplicate_count
-----	-----
Amazon	2

****Query #20****

-- Query 20: Classify employees into salary bands:

```

SELECT e.emp_name AS Employee_Name,e.salary AS Salary,
CASE
    WHEN e.salary < 80000 THEN 'Low'
    WHEN e.salary BETWEEN 80000 AND 120000 THEN 'Medium'
    ELSE 'High'
END AS Salary_band
FROM employees e;

```

Employee_Name	Salary	Salary_band
-----	-----	-----
John	120000.0	Medium
Mike	95000.0	Medium
David	90000.0	Medium
Sarah	140000.0	High
Priya	110000.0	Medium
Ravi	105000.0	Medium
Anil	85000.0	Medium
Emma	125000.0	High
Robert	85000.0	Medium

Sophia	95000.0	Medium	
Chris	65000.0	Low	
James	130000.0	High	
Linda	100000.0	Medium	
Kevin	95000.0	Medium	
Nancy	75000.0	Low	

****Query #21****

-- Bonus Interview Questions:

-- Query 21: Find employees having the same salary.

SELECT e1.emp_name, e1.salary

FROM employees e1

JOIN employees e2 ON e1.salary = e2.salary

AND e1.emp_id <> e2.emp_id

ORDER BY e1.salary;

emp_name	salary	
----------	--------	--

-----	-----	
-------	-------	--

Robert	85000.0	
--------	---------	--

Anil	85000.0	
------	---------	--

Kevin	95000.0	
-------	---------	--

Sophia	95000.0	
--------	---------	--

Kevin	95000.0	
-------	---------	--

Mike	95000.0	
------	---------	--

Sophia	95000.0	
--------	---------	--

Mike	95000.0	
------	---------	--

****Query #22****

-- Query 22: Find customers who placed more than 3 orders.

```
SELECT o.customer_id, COUNT(*) AS order_count
FROM orders o
GROUP BY o.customer_id
HAVING COUNT(*) > 3;
```

customer_id	order_count
1	5
2	5

****Query #23****

-- Query 23: Find the month with highest sales.

```
SELECT MONTH(order_date) AS order_month,
       SUM(amount) AS total_sales
FROM orders
GROUP BY MONTH(order_date)
ORDER BY total_sales DESC
LIMIT 1;
```

order_month	total_sales
4	130000.0

****Query #24****

-- Query 24: Calculate days between consecutive orders for each customer.

```
SELECT o.customer_id, o.order_id, o.order_date,
       DATEDIFF(
         o.order_date,
         LAG(o.order_date) OVER (PARTITION BY o.customer_id ORDER BY o.order_date)
       ) AS days_between_orders
FROM orders o;
```

customer_id	order_id	order_date	days_between_orders
-----	-----	-----	-----
1	1001	2024-01-05	
1	1003	2024-02-02	28
1	1007	2024-04-10	68
1	1013	2024-07-01	82
1	1018	2024-09-20	81
2	1002	2024-01-10	
2	1005	2024-03-01	51
2	1011	2024-06-10	101
2	1015	2024-08-10	61
2	1020	2024-10-22	73
3	1004	2024-02-15	
3	1009	2024-05-05	80
3	1016	2024-08-15	102
4	1006	2024-03-18	

4	1014	2024-07-08	112	
5	1008	2024-04-25		
5	1012	2024-06-15	51	
5	1017	2024-09-01	78	
6	1010	2024-05-20		
6	1019	2024-10-05	138	

****Query #25****

-- Query 25: Find departments where no employee was hired in the last 2 years.

```

SELECT d.dept_id, d.dept_name
FROM departments d
WHERE d.dept_id NOT IN (
  SELECT DISTINCT e.dept_id
  FROM employees e
  WHERE e.hire_date >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
);

```

dept_id	dept_name	
-----	-----	
1	Engineering	
2	Data	
3	Finance	
4	HR	
5	Sales	

[View on DB Fiddle](<https://www.db-fiddle.com/>)